

How to Make AI: Building a Language Model From Scratch

Ardit · Grow Labs, Amaro Than Platforms · 21 September 2026

Abstract

This paper explains, step by step, how a language model is built, using our own work at Grow Labs as the example. We trained a 20-million-parameter model on 10,000 samples on a single Apple M1 Max laptop. We describe each stage: preparing data, building the model, training it, tuning hyperparameters, measuring overfitting and generating text. We report what worked, what failed and why. Our plan is to scale this pipeline to 12 billion parameters and, later, toward 1 trillion.

1. What an AI language model is

A language model is a program that reads text and predicts what comes next. It does this one **token** at a time. A token is a word or a piece of a word.

Inside, the model is a large set of numbers called **parameters** (or weights). Our model has about 20 million of them. Training means adjusting these numbers until the model's predictions become good.

The whole process has six stages:

- 1 **Collect data:** the text the model will learn from.
- 2 **Prepare data:** clean it and put it in a format the program can read.
- 3 **Tokenize:** turn text into numbers.
- 4 **Build the model:** choose its size and structure.
- 5 **Train:** show it the data many times and correct its mistakes.
- 6 **Test and generate:** check how well it learned, then let it write.

2. Step 1: Data

Data is what the model learns from. If a fact is not in the data, the model cannot know it.

Stage	Samples	Content
First dataset	5,292	Machine learning concepts, Apple knowledge, Universe facts, Grow Lab information
Added later	About 5,000	More samples on the same topics, including public datasets
Total	About 10,000	Used for the current model

Format. We store data as **JSONL**: one JSON object per line. Each line is one training example. This makes the data easy to read, count and split. An example line:

```
{"prompt": "What is overfitting?", "response": "Overfitting is when a model memorises..."}
```

Train and validation split. We keep a small part of the data hidden from training. This is the **validation set**. We use it to check whether the model truly learned or only memorised.

3. Step 2: Tokenization

Computers work with numbers, not letters. A **tokenizer** splits text into tokens and gives each one an ID number. For example, "learning" might become two tokens, "learn" and "ing". The model only ever sees these numbers. When it writes, it outputs token IDs, and the tokenizer turns them back into text.

4. Step 3: The model

Our model is a small **transformer**, the same basic design used by large models such as GPT and Claude. It has three main parts:

- **Embeddings:** turn each token ID into a list of numbers that carries its meaning.
- **Transformer layers:** use *attention* to let each token look at the tokens before it and decide which ones matter.
- **Output layer:** gives a probability for every possible next token.

The model predicts one token, adds it to the text, and repeats. This is how it writes full sentences.

5. Step 4: Training

Training is a loop. The model makes a prediction, we measure how wrong it is, and we adjust the weights slightly to make it less wrong.

The measure of wrongness is called **loss**. We use cross-entropy loss:

$$Loss = - (1/N) \sum \log P(\text{correct next token})$$

If the model gives the correct token a high probability, the loss is low. If it gives it a low probability, the loss is high. Training tries to make the loss as low as possible.

- **Step:** one update of the weights, using one small group of examples (a batch).
- **Epoch:** one full pass through all the training data.
- **Hardware:** all training so far runs on an Apple M1 Max laptop.

6. Step 5: Hyperparameters

Hyperparameters are the settings we choose before training. They are not learned by the model. Getting them right was the biggest lesson of our work.

Setting	What it does	Our value
Learning rate	How big each weight update is	0.0001
Batch size	How many examples per step	4
Dropout	Randomly switches off parts of the model during training, so it cannot rely on memorising	0.15
Weight decay	Keeps weights small, which reduces memorising	0.02
Epochs	How many times the model sees all the data	100
Temperature	How random the output is when generating text (not used in training)	0.25

7. Step 6: Measuring overfitting

Overfitting means the model memorises the training data instead of learning general patterns. It then does well on data it has seen and badly on new data.

We measure this with the **overfitting gap**: the difference between validation loss and training loss.

$$\text{Gap} = \text{Validation loss} - \text{Training loss}$$

A small gap means the model performs about the same on new data as on training data, so it generalises. A large gap means it memorised. We treat a gap under 1.0 as good generalisation.

Phase	Data	Gap	What it meant
Early phase	Small dataset	2.9+	Strong overfitting. This was intentional, to confirm the pipeline could learn at all.
Second run	5,000 samples	0.5788	Good generalisation
Current run	10,000 samples	0.1851	Excellent generalisation. Doubling the data cut the gap by about 68%.

Key finding: more data was the strongest single fix for overfitting in our tests. But a low gap is not enough on its own. The model still needs good hyperparameters to avoid hallucinations and repeating loops when it writes.

8. What went wrong, and what we learned

Problem	Cause	Fix
Training became unstable and output looped	Learning rate too high (0.05)	Lower it to 0.0001
Model made things up (hallucinations)	Dropout and weight decay too low	Dropout 0.15, weight decay 0.02
Output repeated the same words	Temperature too low (0.1)	Raise to 0.25
Output became random and invented facts	Temperature too high (0.8)	Lower to 0.25

Temperature needs a short explanation. When generating, the model gives a probability to every possible next token. Low temperature makes it always pick the most likely one, which leads to repetition. High temperature makes it pick less likely tokens more often, which leads to nonsense. For our model, 0.25 was the best balance.

9. Next steps

- **Retrain the 10,000-sample model** with the corrected settings (learning rate 0.0001, dropout 0.15, weight decay 0.02) for about 250,000 steps. We estimate 40 to 50 hours on the M1 Max.
- **Test for hallucinations:** check whether the new settings remove made-up answers and loops.
- **Move to cloud GPUs:** we have \$290 of Google Cloud credits for A100 or H100 GPUs.
- **Scale up:** M1 Max (20M) → supercomputer (12B) → long-term goal of 1 trillion parameters.

Reality check on scale. A common rule for estimating training cost is $\text{compute} \approx 6 \times \text{parameters} \times \text{tokens}$. A widely used guideline (Hoffmann et al., 2022) suggests about 20 training tokens per parameter. The figures below are rough estimates, not measurements.

Model size	Tokens needed (approx.)	A100 GPU-hours (approx.)
20M (current)	0.4 billion	Under 1
12B	240 billion	About 30,000
1T	20 trillion	About 200 million

The \$290 in credits covers roughly 80 A100 hours. That is enough for experiments with models of a few hundred million parameters, but not for the 12B stage. The 12B stage will need a compute partner, a research grant or supercomputer access.

10. Conclusion

Building an AI model follows six clear stages: data, preparation, tokenization, model design, training and testing. Our work shows that this can be done on one laptop at small scale. The most important lessons were that more data reduces overfitting the most, and that learning rate, dropout, weight decay and temperature decide whether the model writes useful text or loops and hallucinates. The same pipeline is the foundation for every larger model we plan to build.

References

- Vaswani, A. et al. (2017). Attention Is All You Need.
- Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting.
- Kaplan, J. et al. (2020). Scaling Laws for Neural Language Models.
- Hoffmann, J. et al. (2022). Training Compute-Optimal Large Language Models.
- Holtzman, A. et al. (2020). The Curious Case of Neural Text Degeneration.